

第16回 前期の確認と JavaScript の基本 1

実習時のフォルダ構成

次のフォルダ構成で作業する。`名前` の部分は、自分の名前にする。

```
Web_名前
├── 16
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       ├── style.css
│       ├── main.js
│       └── images
│           └── 画像ファイル
```

- `sample` : 講義で使うサンプル (第 2 部の講義内容)
- `exercise` : 演習で作るファイル (演習 1 ~ 3)

第1部 前期の確認 (HTML / CSS)

演習 1 ゲーム紹介ページを作る

前期に学んだ HTML / CSS を、どれくらい書けるか確認する。`exercise` フォルダに `index.html` と `style.css` を作り、次の 2 段階で「ゲーム紹介ページ」を作る。途中で分からなくなったら、その段階で止まってよい (どこまで書けたかを確認するための演習)。余裕がある人は、最後の「発展 (任意)」に取り組む。

完成イメージ (画面の並び) :

```
+-----+
| ゲーム紹介                ← header |
+-----+
|  おすすめのゲーム        ← main  |
|  文章、画像、箇条書き、リンク  |
|                                |
|  +-----+                |
|  | お知らせ              | ← aside |
|  +-----+                |
+-----+
| © 2026 Game Club          ← footer |
+-----+
```

演習 1 - 1 HTML を書く

- HTML の基本構造を書く (`<!DOCTYPE html>`、`<html lang="ja">`、`<head>`、`<body>`)。

- `<head>` に、文字コード、viewport、`<title>`、CSS ファイル (`style.css`) の読み込みを書く。
- `exercise` フォルダの中 (`index.html` と同じ場所) に `images` フォルダを作り、画像ファイルを 1 つ入れる。
- `<body>` に、次の要素を書く。
- `<header>` : ページのタイトル (`<h1>`)。 `<h1>` に `id="title"` を付ける。
- `<main>` : 見出し (`<h2>`)、段落 (`<p>`)、画像 (`<img>`)、箇条書き (`<ul>` と `<li>` を 2 つ以上)、リンク (`<a>`)。
 - `<img>` の `src` 属性に、`images` フォルダ内の画像を相対パスで指定する。
 - `<img>` の `alt` 属性に、画像の説明を書く。
- `<aside>` : 見出し (`<h2>`) と段落 (`<p>`)。 `class="side"` を付ける。
- `<footer>` : 段落 (`<p>`)
- 段落の中の一部を `<span>` で囲み、`class="point"` を付ける。
- `<main>` と `<aside>` を、`<div class="container">` で囲む。

演習 1-2 CSS で見た目を整える

- 次の 3 種類のセレクタを、それぞれ 1 回以上使う。
- 要素セレクタ (例: `body`、`header`、`footer`、`a`)
- クラスセレクタ (`.point`、`.side`、`.container`)
- ID セレクタ (`#title`)
- `body` : 余白を 0 にし、フォントと文字色を指定する。
- `header` : 背景色、文字色、`padding` を指定する。
- `#title` : `margin` を 0 にし、文字の大きさを指定する。
- `.point` : 文字色と太さを指定する。
- `.container` : `max-width`、`margin` (左右を自動)、`padding` を指定し、ページの中央に寄せる。
- `.side` : `width`、`padding`、`border`、背景色を指定する。
- `footer` : 文字を中央に寄せ、背景色と `padding` を指定する。

発展 (任意)

必須の演習が終わった人は、次に取り組む。

- `box-sizing` : `.side` に `box-sizing: border-box;` を指定し、`width` に `padding` と `border` を含めた幅にする。
- レスポンシブ対応 (`@media`) : 画面幅が 600px 以下のときだけ、次のように変わるようにする。
- `#title` の文字を小さくする。
- `.side` の幅を 100% にする。
- `:hover` と `transition` : `.side` にマウスを乗せると、背景色が 0.3 秒かけて変わるようにする。
- 2 カラム : `<main>` と `<aside>` を横に並べる。前期で習った方法 (`float` か `Flexbox`) で書いてよい。
- `table` : `<table>` を使って、ゲームの一覧表を作る (`<tr>`、`<th>`、`<td>`)。

講義内容（解説・復習）

演習 1 で書いた内容を、順に振り返る。

1. HTML の基本構造

HTML は、ページの「構造」と「内容」を書く言語。すべてのページは次の骨組みから始まる。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ゲーム紹介</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- ここに表示する内容を書く -->
</body>
</html>
```

- `<head>` にはページの設定（文字コード、タイトル、CSS の読み込みなど）を書く。画面には表示されない。
- `<body>` に画面に表示する内容を書く。
- `<meta charset="UTF-8">` がないと、日本語が文字化けすることがある。

2. よく使うタグ

タグ	意味
<code><h1> ~ <h6></code>	見出し（ <code>h1</code> が最上位の見出し）
<code><p></code>	段落
<code></code>	リンク
<code></code>	画像
<code> </code>	箇条書き（ <code>ul</code> ：順序なし、 <code>ol</code> ：番号付き）
<code><div> </code>	まとめり・一部分を囲む（ <code>div</code> はブロック、 <code>span</code> は文字の一部）
<code><header> <main> <aside> <footer></code>	ページの役割ごとの区切り

- `class` と `id` は、CSS で「どの要素を装飾するか」を指定するための名前。`class` は複数の要素に付けられる。同じ `id` の値は、1 つのページ内で重複させない。

3. CSS の書き方

CSS は、見た目を整える言語。`セレクタ { プロパティ: 値; }` の形で書く。

```
body {
  margin: 0;
  font-family: sans-serif;
  color: #333333;
}

header {
  background-color: #2c3e50;
  color: #ffffff;
  padding: 16px;
}

#title {
  margin: 0;
  font-size: 28px;
}

.point {
  color: #e74c3c;
  font-weight: bold;
}
```

セレクトタ	書き方	対象
要素セレクトタ	<code>header { }</code>	その名前のタグすべて
クラスセレクトタ	<code>.point { }</code>	<code>class="point"</code> の要素
ID セレクトタ	<code>#title { }</code>	<code>id="title"</code> の要素

- HTML の `<head>` に `<link rel="stylesheet" href="style.css">` を書いて、CSS ファイルを読み込む。
- よく使うプロパティ: `color` (文字色)、`background-color` (背景色)、`font-size` (文字の大きさ)、`font-weight` (太さ)、`text-align` (文字の配置)。

4. ボックスモデル

すべての要素は、四角い箱として扱われる。内側から順に、内容 (`width`・`height`) → `padding` (内側の余白) → `border` (枠線) → `margin` (外側の余白)。

```
.side {
  width: 300px;
  box-sizing: border-box;
  padding: 16px;
  border: 1px solid #cccccc;
  background-color: #f5f5f5;
}
```

- `box-sizing: border-box;` を付けると、`width` に `padding` と `border` を含めた幅になる。付けないと、見た目の幅が `width` より大きくなる。
- `margin: 0 auto;` は、上下 0・左右を自動にする指定。幅を決めた要素を中央に寄せるときに使う。

```
.container {
  max-width: 900px;
  margin: 0 auto;
  padding: 20px;
}
```

- `max-width`：幅の上限。画面が狭いときは、画面の幅に合わせて縮む。

5. レスポンシブ対応

画面の幅に合わせて、見た目を切り替える。

```
@media (max-width: 600px) {
  #title {
    font-size: 22px;
  }

  .side {
    width: 100%;
  }
}
```

- `@media (max-width: 600px)`：画面幅が 600px 以下のときだけ、中の CSS が有効になる（スマートフォンで正しい幅にするため、HTML の `<meta name="viewport" ...>` も必要）。

6. CSS アニメーション

`transition` を使うと、状態の変化がなめらかになる。

```
.side {
  transition: background-color 0.3s;
}

.side:hover {
  background-color: #e8f4fd;
}
```

- `:hover` はマウスが乗っている間の状態、`transition: プロパティ 時間;` はその変化にかかる時間。

完成例（答え合わせ用）

演習 1 の完成例。自分のページと見比べる。この完成例には、必須の内容に加えて、発展（任意）の `box-sizing`、レスポンシブ対応（`@media`）、`:hover` と `transition` も含まれている（2 カラムと `table` は含まれていない）。

`index.html`

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ゲーム紹介</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1 id="title">ゲーム紹介</h1>
  </header>

  <div class="container">
    <main>
      <h2>おすすめのゲーム</h2>
      <p>今回は<span class="point">アクションゲーム</span>を紹介します。</p>
      
      <ul>
        <li>ゲームA</li>
        <li>ゲームB</li>
      </ul>
      <p><a href="https://example.com/">詳しくはこちら</a></p>
    </main>

    <aside class="side">
      <h2>お知らせ</h2>
      <p>更新情報をここに書きます。</p>
    </aside>
  </div>

  <footer>
    <p>&copy; 2026 Game Club</p>
  </footer>
</body>
</html>
```

style.css

```
body {
  margin: 0;
  font-family: sans-serif;
  color: #333333;
}

header {
  background-color: #2c3e50;
  color: #ffffff;
  padding: 16px;
}

#title {
  margin: 0;
  font-size: 28px;
}

.point {
  color: #e74c3c;
  font-weight: bold;
}

a {
  color: #2980b9;
}

.container {
  max-width: 900px;
  margin: 0 auto;
  padding: 20px;
}

.side {
  width: 300px;
  box-sizing: border-box;
  padding: 16px;
  border: 1px solid #cccccc;
  background-color: #f5f5f5;
  transition: background-color 0.3s;
}

.side:hover {
  background-color: #e8f4fd;
}

footer {
  padding: 16px;
  text-align: center;
  background-color: #eeeeee;
}

@media (max-width: 600px) {
  #title {
    font-size: 22px;
  }

  .side {
```

```
width: 100%;  
}  
}
```

第2部 JavaScript の基本 1

講義内容

1. JavaScript とは

技術	役割	たとえば
HTML	ページの構造・内容を書く	骨組み
CSS	見た目を整える	服装・装飾
JavaScript	動き・処理を加える	動作・頭脳

- ブラウザの中で動くプログラミング言語。ボタンを押したときの反応、計算、アニメーション、ゲームなどを作れる。
- **Java とは別の言語**（名前が似ているだけ）。
- 後期の最終目標は、JavaScript と Canvas を使った Web ゲーム制作。

2. JavaScript を書く場所

JavaScript は HTML の中に直接書くこともできるが、この授業では別ファイル（`main.js`）に書き、HTML から読み込む。HTML の `</body>` の直前に、次の 1 行を書く。

```
<script src="main.js"></script>  
</body>
```

- この授業では、まず `</body>` の直前に書く方法を使用する。先に HTML が読み込まれてから JavaScript を動かすため。
- `main.js` は `index.html` と同じフォルダに置く（パスが違うと動かない）。
- 講義のコードは `sample` フォルダの `main.js` に書いて確認する。

講義では、`sample` フォルダに次の 2 つのファイルを作って使う。

```
sample  
├─ index.html  
└─ main.js
```

`sample/index.html` には、次の内容を書く。`main.js` を読み込んで、ブラウザで実行できる最小限の HTML である。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本1</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

この後の `console.log` などの JavaScript のコードは、すべて `sample/main.js` に書く。`main.js` を保存したら、`sample/index.html` をブラウザで開いて、結果を確認する。

3. console.log と開発者ツール

プログラムの結果を確認する最も基本的な方法が `console.log`。ブラウザの開発者ツール（F12 キー）の「コンソール」タブに表示される。

```
console.log("Hello, JavaScript!");
console.log(1 + 2);
```

- この授業では、文の終わりにセミコロン `;` を付けて書く。文字列は `" "` で囲む。
- `//` から行末まではコメント（実行されない）。
- エラーが出たら、コンソールに赤字でエラー内容と行番号が表示される。まずここを読む。

4. 変数

値に名前を付けて保管しておく入れ物が「変数」。ゲームなら HP や攻撃力、スコアなどを入れる。計算は算数と同じ記号（`+` `-` `*` `/`）を使う。

```
const maxHp = 100;           // 変更しない値（最大HP）
let hp = 100;                // 変更する値（現在のHP）

hp = hp - 30;                // ダメージを受けて、現在のHPが減る
console.log(maxHp, hp);      // 100 70

// const の変数を書き換えるとエラーになる
// maxHp = 200;              // エラー: Assignment to constant variable.
```

- `let`：値を書き換える予定のもの（HP、スコアなど）。
- `const`：書き換えないもの（キャラクター名、最大値など）。迷ったら `const` にして、必要になったら `let` に直す。
- 変数名は、何を入れるか分かる名前にする（例：`maxHp`、`hp`）。
- 変数名のルール：英数字と `_` が使える／数字で始めない／予約語（`let`、`const` など）は使えない／大文字と小文字は区別される。
- `=` は「等しい」ではなく「右の値を左の変数に入れる」という意味。
- `console.log(maxHp, hp);` のように、`,` で区切ると、複数の値を 1 行に表示できる。

演習 2 自己紹介を console に表示する

`exercise` フォルダの `index.html` (途中まででもよい) に、`main.js` を読み込む。同じ `exercise` フォルダに `main.js` を作成して、次を行う。

- 自分の名前と好きなゲームを、それぞれ `const` の変数に入れる (変数名の例: `userName`、`favoriteGame`)。
- `console.log` で、それぞれの値を表示する。
- 開発者ツール (F12) を開き、コンソールに表示されることを確認する。
- `favoriteGame` に別の値を代入する行を書き、エラーが出ることを確認する。エラーの内容を読んだら、その行の先頭に `//` を付けてコメントにする。

演習 3 ダメージ計算

`main.js` に続けて、次を行う。

- 敵の名前、敵の HP (例: 120)、こちらの攻撃力 (例: 35) を、変数に入れる。書き換える変数は `let`、書き換えない変数は `const` にする (変数名の例: `enemyName`、`enemyHp`、`attackPower`)。
- 攻撃を 1 回行った結果 (HP から攻撃力を引く) を敵の HP に代入し、敵の名前と残り HP を `console.log` で表示する。
- 同じ攻撃をあと 2 回行い、そのつど残り HP を表示する。